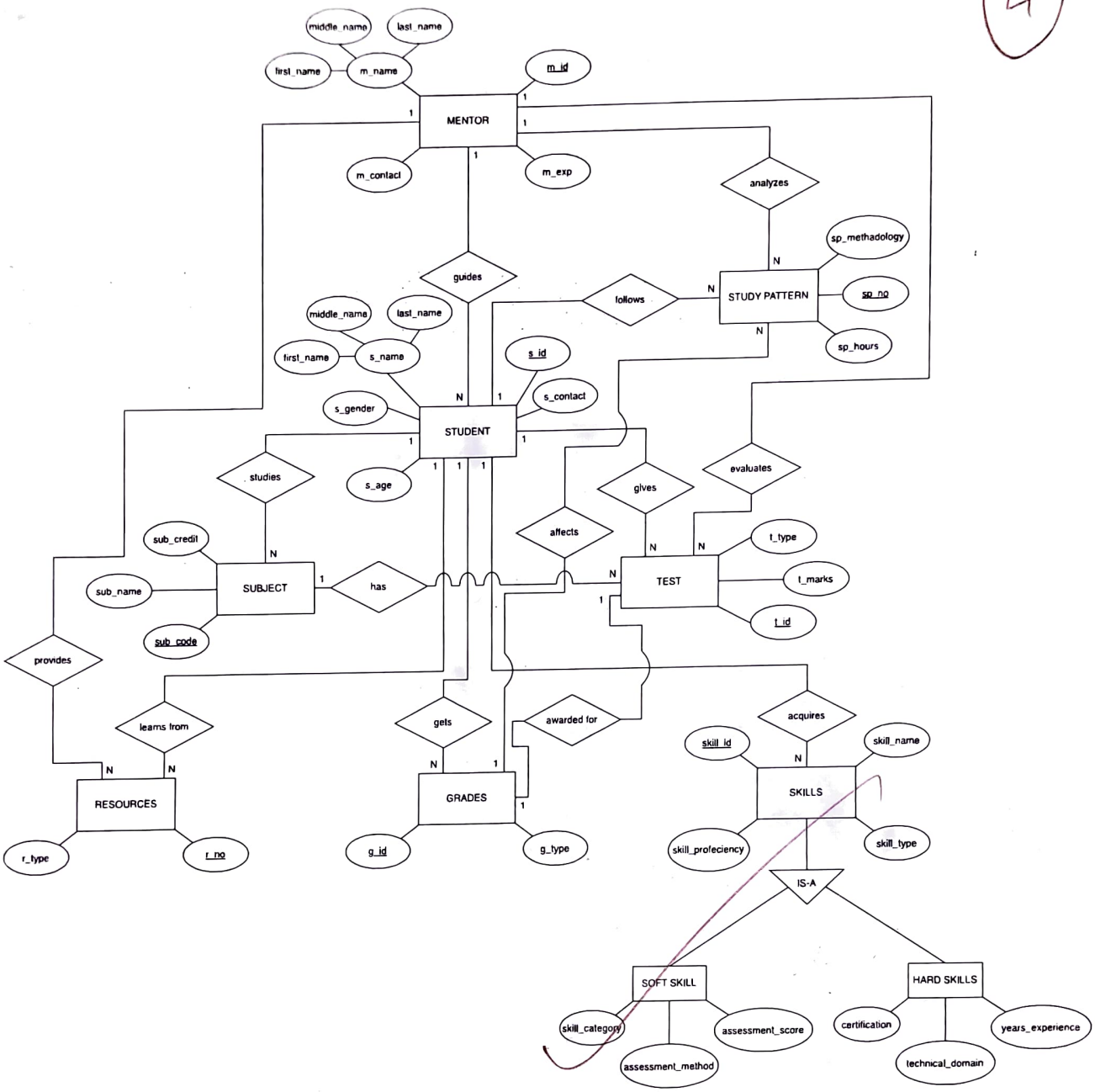
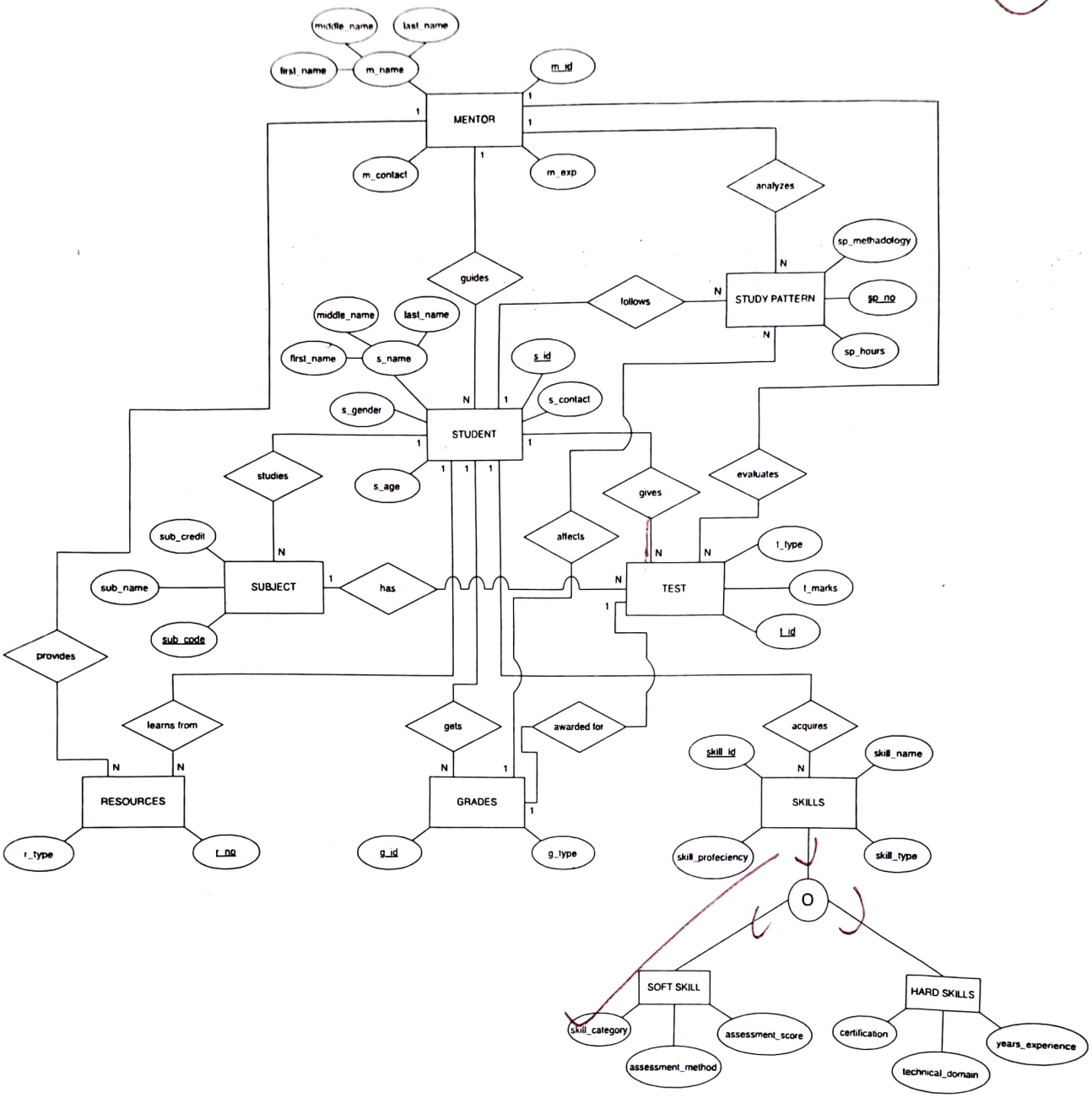

middle_name
last_name
first_name
m_name
m_id
MENTOR
m_contact
m_exp
analyzes
sp_methadology
guides
follows
STUDY PATTERN
sp_no
sp_hours
middle_name
last_name
first_name
s_name
s_id
s_contact
s_gender
STUDENT
studies
s_age
gives
evaluates
affects
sub_credit
sub_name
SUBJECT
has
TEST
t_type
t_marks
t_id
sub_code
provides
learns from
gets
awarded for
acquires
skill_id
skill_name
SKILLS
skill_profeciency
skill_type
RESOURCES
r_type
r_no
GRADES
g_id
g_type
IS-A
SOFT SKILL
skill_category
assessment_score
assessment_method
HARD SKILLS
certification
years_experience
technical_domain


middle_name
last_name
first_name
m_name
m_id
MENTOR
m_contact
m_exp
analyzes
guides
follows
STUDY PATTERN
sp_methadology
sp_no
sp_hours
middle_name
last_name
first_name
s_name
s_id
s_contact
s_gender
STUDENT
s_age
studies
gives
evaluates
affects
sub_credit
sub_name
SUBJECT
has
TEST
t_type
t_marks
t_id
sub_code
provides
learns from
gets
awarded for
acquires
skill_id
skill_name
RESOURCES
GRADES
SKILLS
r_type
r_no
g_id
g_type
skill_proficiency
skill_type
O
SOFT SKILL
HARD SKILLS
skill_category
assessment_score
assessment_method
certification
years_experience
technical_domain

13/8

Name :- Aaryan N. Rasare Roll no :- 5004 Div :- S
batch :- A1 SAP ID :- 60018250070 branch :- AI&DS

Experiment No 2

Aim: Map ER / EER to Relational schema model

Theory:

conversion from ER diagrams to tables

step1 :- Mapping of Regular Entity types:

① For each regular (strong) entity type E in the ER schema create a Relation R that includes all the simple attributes of E. Include only the simple component attributes of a composite, the set of simple attributes that form it will together form the primary key of R

step2 :- Mapping of weak Entity Types

① for each weak entity type W in the ER schema with owner entity type E, create a relation R and include all the attributes of W and the specific attributes of R. In addition include the primary key attributes of the owner entity type. The primary key of R is the combination of the primary key(s) of the owner(s) and the partial key of the weak entity type W.

step 3 :- Mapping of Binary 1 : 1 Relationship types.

for each regular binary 1:1 relationship type R in the

Sundaram
FOR EDUCATIONAL USE

ER schema, identify the relations S and T that correspond to the Entity Types participating in R. Choose one of the relations - S, say - and include as a foreign key in S the primary key of T. It is better to choose an entity type with total participation in R in the role of S. Include all the simple attributes (or simple components of a composite attributes) of the 1:1 relationship type R as attributes of S.

Step 4: Mapping of Binary 1:N relationship types.

For each regular binary 1:N relationship type R, identifying the relation S that represent the participating Entity type at the N side of the relationship. Include as foreign key in S the primary key of T that represents the other entity type participating in R; this is done because each entity instance of the N side is related to at most one entity instance of the 1-side of the relationship type. Include any simple attributes (or simple components of composite composite attributes) of the 1:N relationship type as attributes of S

Step 5: Mapping of Binary M:N relationship Types

For each binary M:N relationship Type R, create a new relation S to represent R, include the primary Key

FOR EDUCATIONAL USE

Sundaram

of the participating entity types; their combination will form the primary key of S. Also include any simple attributes of the M:N relationship types (or simple components of a composite attributes) as attributes of S.

step 6: Mapping of Multivalued attributes:

For each Multivalued attribute A, create a new relation R. This relation R will include an attribute corresponding to A, plus the primary key attribute K - of the relation that represent the entity type or relationship type that has A as an attribute. The primary key of R is the combination of A and K. If the multivalued attribute is composite, we include its simple components.

step 7: Mapping of N-ary Relationship types:

For each N-ary relationship type R, where N > 2, create a new relation S to represent R. Include the primary keys of the relationship that represent the participating entity types. Also include any simple attributes of the N-ary relationship types (or simple components of a composite attributes) as attributes of S.

Sundaram
FOR EDUCATIONAL USE

Mapping EER model constructs to Relations

Step 8: options for Mapping Specialization or Generalization

Convert each specialization with m subclasses {S1, S2, ..., Sm} and generalized superclass C where the attributes of C are {k, a1, ... an} and k is the (primary) key, into relational schemas using one of the four options:

option 8A :- Multiple relations - superclass and subclasses

Create a relation L for C with attributes Attrs(L) = {k, a1, ... an} and PK(L) = k. Create a relation Li for each subclass Si, 1<i<m with the attributes Attrs(Li) = {k} ∪ {attributes of Si} and PK(Li) = k. This option works for any specialization (total or partial, disjoint or overlapping).

option 8B :- Multiple relations - Subclass relations only

Create a relation Li for each subclass Si, 1<i<m, with the attributes Attr(Li) = {attributes of Si} ∪ {k, a1, ..., an} and PK(Li) = k. This option only works for a specialization whose subclasses are total (every entity in the superclass must belong to (at least) one of the subclasses

Sundaram
FOR EDUCATIONAL USE

If the specialization is overlapping an entity may be duplicate in several relations.

Option 8C: single relation with one type attribute

create a single relation L with attributes Attrs(L) = {K, a1, ..., an} U of attributes of S1} U .... U {attributes of Sm} U {t} and PK(L) = k. the attribute t is called a type (or discriminating) attribute that indicates the subclass to which each tuple belongs (Disjoint)

Option 8D: single relation with multiple type attributes

create a single relation schema L with attributes Attrs(L) = {K, a1, ... an} U {attributes of S1} U .... U {attributes of Sm} U {t1, t2, ... tm} and PK(L) = k. each ti, 1 < i < m, is a Boolean type attribute indicating whether a tuple belongs to the subclass Si (overlap)

Step 9: Mapping of Union Types (categories)

- For mapping a category whose defining superclasses have different keys, it is customary to specify a new key attribute, called a surrogate key, when creating a relation to correspond to the category.

FOR EDUCATIONAL USE

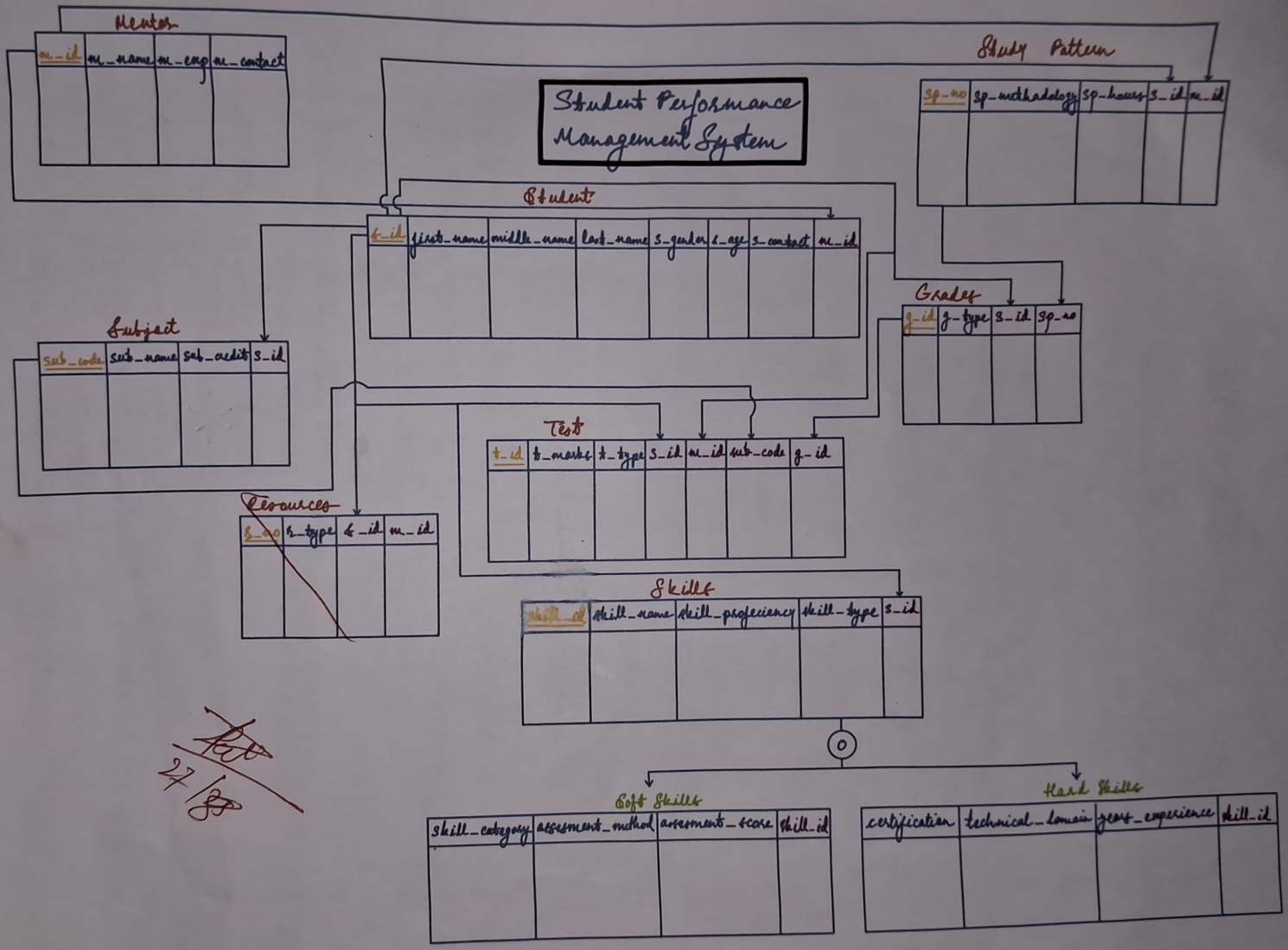
Sundaram®

- For Example we can create a relation owner to correspond to the owner category and include any attributes of the category in this relation

Conclusion:

Thus we have studied the mapping of ER to relational table.

Sundaram FOR EDUCATIONAL USE


Student Performance Management System
Mentor
m_id
m_name
m_exp
m_contact
Study Pattern
sp_no
sp_methodology
sp_hours
s_id
m_id
Student
s_id
first_name
middle_name
last_name
s_gender
s_age
s_contact
m_id
Subject
sub_code
sub_name
sub_credit
s_id
Grades
g_id
g_type
s_id
sp_no
Test
t_id
t_marks
t_type
s_id
m_id
sub_code
g_id
Resources
r_no
r_type
t_id
m_id
Skills
skill_id
skill_name
skill_proficiency
skill_type
s_id
Soft Skills
skill_category
assessment_method
assessment_score
skill_id
Hard Skills
certification
technical_domain
years_experience
skill_id

	Field	Type	Null	Key	Default	Extra
▶	s_id	int	NO	PRI	NULL	
	s_name	varchar(30)	NO		NULL	
	s_gender	char(1)	YES		NULL	
	s_age	int	YES		NULL	
	m_id	int	YES		NULL	

	Field	Type	Null	Key	Default	Extra
▶	m_id	int	NO	PRI	NULL	
	m_first_name	varchar(30)	NO		NULL	
	m_middle_name	varchar(30)	NO		NULL	
	m_last_name	varchar(30)	NO		NULL	
	m_contact	int	YES		NULL	
	m_exp	int	YES		NULL	

	Field	Type	Null	Key	Default	Extra
▶	sub_code	varchar(15)	NO	PRI	NULL	
	sub_name	varchar(15)	YES		NULL	
	sub_credit	float	YES		NULL	
	s_id	int	YES		NULL	

	Field	Type	Null	Key	Default	Extra
▶	r_no	int	NO	PRI	NULL	
	r_type	varchar(15)	YES		NULL	
	sub_code	varchar(15)	YES		NULL	
	s_id	int	YES		NULL	
	m_id	int	YES		NULL	

	Field	Type	Null	Key	Default	Extra
▶	sp_no	int	NO	PRI	NULL	
	sp_methodology	varchar(20)	YES		NULL	
	sp_hours	int	YES		NULL	
	s_id	int	YES		NULL	
	m_id	int	YES		NULL	

	Field	Type	Null	Key	Default	Extra
▶	t_id	int	NO	PRI	NULL	
	t_type	varchar(20)	YES		NULL	
	t_marks	int	YES		NULL	
	sub_code	varchar(15)	YES		NULL	
	s_id	int	YES		NULL	
	m_id	int	YES		NULL	

	Field	Type	Null	Key	Default	Extra
▶	g_id	int	NO	PRI	NULL	
	g_type	varchar(20)	YES		NULL	
	s_id	int	YES		NULL	
	t_id	int	YES		NULL	

	Field	Type	Null	Key	Default	Extra
▶	skill_id	int	NO	PRI	NULL	
	skill_name	varchar(20)	YES		NULL	
	skill_type	varchar(20)	YES		NULL	
	skill_proficiency	int	YES		NULL	
	s_id	int	YES		NULL	

	Field	Type	Null	Key	Default	Extra
▶	skill_id	int	YES		NULL	
	technical_domain	varchar(20)	YES		NULL	

	Field	Type	Null	Key	Default	Extra
▶	skill_id	int	YES		NULL	
	skill_category	varchar(20)	YES		NULL	

```
create database AARYAN_RAORANE_S004;

use AARYAN_RAORANE_S004;

create table student
(s_id int primary key,
s_name varchar(30) not null,
s_gender char(1) check (s_gender='M' or s_gender='F'),
s_age int,
m_id int
);

describe student;
```

```
create table mentor
(m_id int primary key,
m_first_name varchar(30) not null,
m_middle_name varchar(30) not null,
m_last_name varchar(30) not null,
m_contact int,
m_exp int
);

describe mentor;

create table subject
(sub_code varchar(15) primary key,
sub_name varchar(15),
sub_credit float,
s_id int
);

describe subject;

create table resources
(r_no int primary key,
r_type varchar(15),
sub_code varchar(15),
s_id int,
m_id int
);

describe resources;

create table study_pattern
```

```
(sp_no int primary key,
sp_methodology varchar(20),
sp_hours int,
s_id int,
m_id int
);

describe study_pattern;

create table test
(t_id int primary key,
t_type varchar(20),
t_marks int,
sub_code varchar(15),
s_id int,
m_id int
);

describe test;

create table grades
(g_id int primary key,
g_type varchar(20),
s_id int,
t_id int
);

describe grades;

create table skills
(
```

```
skill_id int primary key,
skill_name varchar(20),
skill_type varchar(20),
skill_proficiency int,
s_id int
);

describe skills;

create table hardskills
(
skill_id int,
technical_domain varchar(20)
);

describe hardskills;

create table softskills
(
skill_id int,
skill_category varchar(20)
);

describe softskills;
```

```
64  select * from mentor
65
```

Result Grid | Filter Rows: | Edit: | Export/Impor

m_id	m_first_name	m_middle_name	m_last_name	m_contact	m_exp
101	sudhir	pratik	dekhne	9556800157	7
102	devendra	uddav	fadnavis	8856422103	5
NULL	NULL	NULL	NULL	NULL	NULL

```
70  select * from student
```

Result Grid | Filter Rows: | Edit:

s_id	s_name	s_gender	s_age	m_id
101	doraemon	M	20	101
NULL	NULL	NULL	NULL	NULL

```
72  insert into subject values(
73  101,'DBMS',4,101
74  )
75
76  select * from subject
```

Result Grid | Filter Rows:

sub_code	sub_name	sub_credit	s_id
101	DBMS	4	101
NULL	NULL	NULL	NULL

```
85  select * from resources
```

Result Grid | Filter Rows: | Edit

r_no	r_type	sub_code	s_id	m_id
101	DBMS Notes	101	101	101
NULL	NULL	NULL	NULL	NULL

```
91 • select * from study_pattern;
```

Result Grid | Filter Rows: | Edit:

sp_no	sp_methodology	sp_hours	s_id	m_id
101	Rot learning	12	101	101
NULL	NULL	NULL	NULL	NULL

```
100 • select * from test;
```

Result Grid | Filter Rows: | Edit:

t_id	t_type	t_marks	sub_code	s_id	m_id
101	term test 1	12	101	101	101
NULL	NULL	NULL	NULL	NULL	NULL

```
102 • insert into grades values(
103   101,'marks',101,101
104 );
105
106 • select * from grades;
```

Result Grid | Filter Rows: | Edit:

g_id	g_type	s_id	t_id
101	marks	101	101
NULL	NULL	NULL	NULL

```
112 • select * from skills
```

Result Grid | Filter Rows: | Edit:

skill_id	skill_name	skill_type	skill_proficiency	s_id
101	creatives	soft skill	7	101
NULL	NULL	NULL	NULL	NULL

```
114  insert into softskills values(
115  101,'creative working'
116  );
117
118  select * from softskills;
```

Result Grid | Filter Rows: | Export:

skill_id	skill_category
101	creative working

```
124  select * from skills
```

Result Grid | Filter Rows: | Edit:

skill_id	skill_name	skill_type	skill_proficiency	s_id
101	creatives	soft skill	7	101
102	DSA	tech skill	8	101
NULL	NULL	NULL	NULL	NULL

```
130  select * from hardskills;
```

Result Grid | Filter Rows:

skill_id	technical_domain
102	general

m_id	m_first_name	m_middle_name	m_last_name	m_contact	m_exp
101	sudhir	pratik	dekhne	9556800157	7
102	devendra	uddav	Latke	8856422103	5
NULL	NULL	NULL	NULL	NULL	NULL

```
136  delete from mentor
137  where m_id = 102
138
139  select * from mentor;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

m_id	m_first_name	m_middle_name	m_last_name	m_contact	m_exp
101	sudhir	pratik	dekhne	9556800157	7
NULL	NULL	NULL	NULL	NULL	NULL

```
use aaryan_raorane_s004;

alter table student
add foreign key(m_id) references mentor(m_id);

alter table subject
add foreign key(s_id) references student(s_id);

alter table resources
add foreign key(s_id) references student(s_id);

alter table resources
add foreign key(m_id) references mentor(m_id);

alter table resources
add foreign key(sub_code) references subject(sub_code);

alter table study_pattern
add foreign key(s_id) references student(s_id);

alter table study_pattern
add foreign key(m_id) references mentor(m_id);

alter table test
add foreign key(s_id) references student(s_id);

alter table test
add foreign key(m_id) references mentor(m_id);

alter table test
add foreign key(sub_code) references subject(sub_code);
```

```
alter table grades
add foreign key(s_id) references student(s_id);

alter table grades
add foreign key(t_id) references test(t_id);

alter table skills
add foreign key(s_id) references student(s_id);

alter table hardskills
add foreign key(skill_id) references skills(skill_id);

alter table softskills
add foreign key(skill_id) references skills(skill_id);


alter table student
add s_address varchar(30);

alter table student
modify column s_address varchar(50);

alter table student
drop column s_address;

alter table mentor
modify column m_contact varchar(10);


insert into mentor values(
```

```
101,'sudhir','pratik','dekhne',9556800157,7
);

insert into mentor values(
102,'devendra','uddav','fadnavis',8856422103,5
);

select * from mentor;

insert into student values(
101,'doraemon','M',20,101
);

select * from student;

insert into subject values(
101,'DBMS',4,101
);

select * from subject;

insert into resources values(
101,'DBMS Notes','101',101,101
);

select * from resources;

insert into study_pattern values(
101,'Rot learning',12,101,101
);
```

```
select * from study_pattern;

insert into test values(
101,'term test 1',12,101,101,101
);

select * from test;

insert into grades values(
101,'marks',101,101
);

select * from grades;

insert into skills values(
101,'creatives','soft skill','7',101
);

select * from skills;

insert into softskills values(
101,'creative working'
);

select * from softskills;

insert into skills values(
102,'DSA','tech skill','8',101
);

select * from skills;
```

```
insert into hardskills values(
102,'general'
);

select * from hardskills;

select * from mentor;

update mentor
set m_last_name = 'Latke'
where m_id = 102;

select * from mentor;

delete from mentor
where m_id = 102;

select * from mentor;
```

Name :- Aaryan N. Rodrane Roll no :- 5004 Div :- S
batch :- A1 SAP ID :- 60018250070 branch :- AIEDS

Experiment No 3

Aim : create a database using DDL and apply integrity constraints.

Theory :

*) DDL commands :-

1. create Table command :- It defines each column of the table uniquely. Each column has minimum of three attributes, a name, datatype and size.

Syntax :

create table <table name> (<col> <datatype> (<size>), <col2> <datatype> (<size>))

Ex :- create table emp(
empno number(4) primary key,
ename char(10));

2. Modifying the structure of tables :

i) Add new columns

Syntax : alter table <tablename>
add (<new col> <datatype(size), <new col>
datatype(size));

Sundaram
FOR EDUCATIONAL USE

Ex:- alter table emp add (sal number(7, 2));

iii) dropping a column from a table

Syntax:- alter table <tablename>
drop column <col>

Ex:- alter table emp
drop column sal

iii) Modifying existing columns

Syntax:- Alter table <tablename>
drop column <col>

Ex:- OR

alter table <tablename>
alter column (<col> <newdatatype> (<newsize>)

Ex:- alter table emp
modify (ename varchar2(15));

iv) Renaming the tables

Syntax:- Rename <oldtables> to <newtables>

Ex:- rename emp to emp1

v) Destroying structure of tables

Syntax: drop table <tablename>

Eg:- drop table emp;

vi) Integrity constraints:-

constraints are the Business rules which are enforced on the data being stored in a table are called constraints.

Types of Data constraints

1. The Primary Key at column level

Syntax:-

```
CReate table tablename(
columnname1 datatype constraint <constraintname1>
 PRIMARY KEY,
columnname 2 datatype,
columnname 3 datatype)
:
)
```

Sundaram®
FOR EDUCATIONAL USE

2. The primary key defined at table level

```
create table tablename(
columnname1 datatype,
columnname2 datatype,
columnname3 datatype,
Primary key (columnname1, columnname2));
```

3. The foreign key defined at column level

Syntax:

```
create table tablename(
columnname1 datatype references tablename[(columnname)]
[On delete cascade],
columnname2 datatype,
columnname3 datatype,
:
);
```

The table in which foreign key is defined is called foreign table or detail table

The table in which primary key is defined and referenced by foreign key is called primary table or Master table.

On delete cascade is set then delete operation in master table will trigger the delete operation for corresponding records in the detail table.

Sundaram
FOR EDUCATIONAL USE

4. The foreign key defined at table level

Syntax:-

```
create table tablename (
columnname1 datatype,
columnname2 datatype,
columnname3 datatype,
primary key (columnname1, columnname2),
foreign key (columnname2) references tablename2);
```

5. If constraint can be given user:-

defined name, the syntax is:
constraint <constraint name> <constraint definition>

6. The check constraint defined at column level:-

```
create table tablename (
columnname1 datatype check (logical expression),
columnname2 datatype,
columnname3 datatype,
...
)
```

7. The check constraint defined at table level:-

```
create table tablename (
columnname1 datatype,
```

Sundaram®
FOR EDUCATIONAL USE

columnname2 datatype,
columnname3 datatype,
check (logical expression1),
check (logical expression2));

8. The Unique constraint defined at the column level:

create table tablename(
columnname1 datatype Unique,
columnname2 datatype unique,
columnname3 datatype .....);

9. The Unique constraint defined at the table level:

create table tablename(
columnname1 datatype,
columnname2 datatype,
columnname3 datatype,
Unique (columnname1));

10. Not Null constraint defined only at column level:

create table tablename(
columnname1 datatype Not Null,
columnname2 datatype Not Null,
columnname3 datatype,....);

Sundaram
FOR EDUCATIONAL USE

11. default constraint defined only to column level

columnname datatype default <value>

*) The Data Manipulation Language:

(DML) is used to retrieve, insert and modify database information.

Basic DML commands:

1. Insert
2. update
3. delete

1. Insert into:

This is used to add records into a relation

a) Inserting a single record

Syntax:

INSERT INTO <relation/tablename>
VALUES (data_1, data_2, ..., data_n)

Example:

INSERT INTO student
VALUES (1, 'Ravi', 'M Tech', 'Palakol')

Sundaram
FOR EDUCATIONAL USE

2. Update - set - where:

This is used to update the content of a record in a relation.

Syntax:

```
UPDATE relation name SET field name1 = data,
                         field name2 = data,
WHERE field name = data;
```

Example:

```
UPDATE student SET
sname = 'Kumar' WHERE Sno = 1;
```

3. Delete from:

This is used to delete all the records of a relation but it will retain the structure of that relation.

a) Delete - from:

Syntax: Delete from relation name;

example: Delete from std;

Sundaram
FOR EDUCATIONAL USE

b) Delete - From Where:

This is used to delete a selected from a relation

Syntax: Delete from relation_name
Where condition.

Example: Delete from student
where sno = 2

Conclusion:

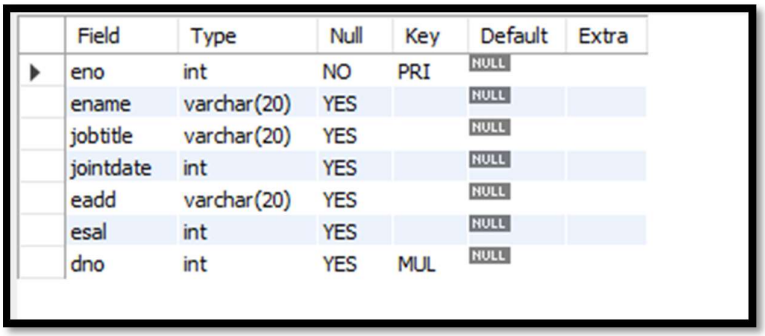
Thus we have implemented DDL and DML commands along with integrity constraints.

Sundaram
FOR EDUCATIONAL USE

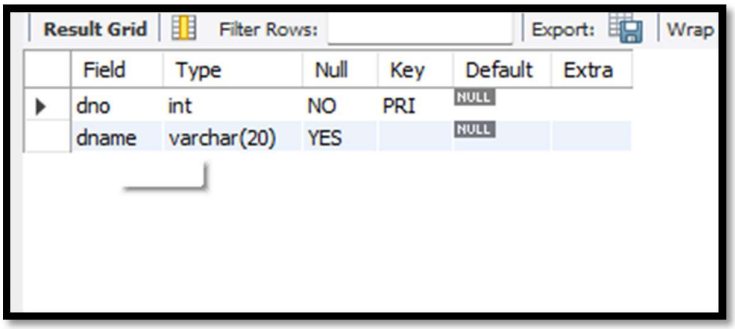
EMPLOYEE (eno, ename, jobtitle, jointdate, eadd, esal, dno)

DEPARTMENT (dno, dname)

1. Display the name of employee whose department name is AIDS
2. Display the name of employee whose jobtitle is manager
3. Retrieve the distinct address from employee
4. List all names and joint date of employee whose name starts with letter m
5. List the names of employee who earn more than rupees 60000
6. List name of all employees who joined between 2023 to 2025
7. List all the names and joint dates of employee whose names has letter e as the 2nd letter
8. Display the name of employee excluding job title manager
9. Display employee in descending order of joint date
10. Retrieve the name of employee who do not get salary
11. Calculate the average salary of dno = 20
12. Retrieve the total no of employee in the department
13. Calculate average salary per department
14. Retrieve maximum and minimum salary offered to employee
15. Give all employee who joined before 2022 a 10% raise



Field	Type	Null	Key	Default	Extra
eno	int	NO	PRI	NULL	
ename	varchar(20)	YES		NULL	
jobtitle	varchar(20)	YES		NULL	
jointdate	int	YES		NULL	
eadd	varchar(20)	YES		NULL	
esal	int	YES		NULL	
dno	int	YES	MUL	NULL	



Result Grid | Filter Rows: | Export: | Wrap

Field	Type	Null	Key	Default	Extra
dno	int	NO	PRI	NULL	
dname	varchar(20)	YES		NULL	

Result Grid | Filter Rows:

dno	dname
10	AIDS
20	Computer Science
30	IT
40	Electronics
50	Mechanical
NULL	NULL

Result Grid | Filter Rows: | Edit: | Expo

eno	ename	jobtitle	jointdate	eadd	esal	dno
101	Manoj	Manager	2024	Mumbai	75000	10
102	Meera	Developer	2021	Pune	65000	20
103	Aakash	Analyst	2023	Delhi	55000	20
104	Neha	Intern	2026	Mumbai	NULL	30
105	Rahul	Manager	2020	Bangalore	90000	40
NULL	NULL	NULL	NULL	NULL	NULL	NULL

1. Display the name of employee whose department name is AIDS:-

```
57 •  select  ename from employee e, department d
58    where e.dno=d.dno AND d.dname='AIDS';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ename
Manoj

2. Display the name of employee whose jobtitle is manager:-

```
60 •  select ename from employee
61    where jobtitle = 'Manager';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ename
Manoj
Rahul

3. Retrieve the distinct address from employee:-

```
63 •  select distinct(eadd) from employee;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conter

eadd
Mumbai
Pune
Delhi
Bangalore

4. List all names and joint date of employee whose name starts with letter m:-

```
65 •  select ename,jointdate from employee
66    where ename like 'm%';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ename	jointdate
Manoj	2024
Meera	2021

5. List the names of employee who earn more than rupees 60000:-

```
68 •  select ename from employee
69    where esal>60000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ename
Manoj
Meera
Rahul

6. List name of all employees who joined between 2023 to 2025:-

```
71 •  select ename from employee
72    where jointdate between 2023 and 2025;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ename
Manoj
Aakash

7. List all the names and joint dates of employee whose names has letter e as the 2nd letter:-

```
74 •  select ename,jointdate from employee
75    where ename like '_e%';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ename	jointdate
Meera	2021
Neha	2026

8. Display the name of employee excluding job title manager:-

```
77 •  select ename from employee
78    where jobtitle != 'Manager';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ename
Meera
Aakash
Neha

9. Display employee in descending order of joint date:-

```
80 •  select * from employee
81    order by jointdate desc;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

eno	ename	jobtitle	jointdate	eadd	esal	dno
104	Neha	Intern	2026	Mumbai	NULL	30
101	Manoj	Manager	2024	Mumbai	75000	10
103	Aakash	Analyst	2023	Delhi	55000	20
102	Meera	Developer	2021	Pune	65000	20
105	Rahul	Manager	2020	Bangalore	90000	40
NULL	NULL	NULL	NULL	NULL	NULL	NULL

10. Retrieve the name of employee who do not get salary:-

```
83 •  select ename from employee
84    where esal is NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ename
Neha

11. Calculate the average salary of dno = 20:-

```
86 •  select avg(esal) from employee
87    where dno=20;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

avg(esal)
60000.0000

12. Retrieve the total no of employee in the department:-

```
89 •  select dno,count(ename) from employee
90    group by dno;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dno	count(ename)
10	1
20	2
30	1
40	1

13. Calculate average salary per department:-

```
92 •  select dno,avg(esal) from employee
93    group by dno;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dno	avg(esal)
10	75000.0000
20	60000.0000
30	NULL
40	90000.0000

14. Retrieve maximum and minimum salary offered to employee:-

```
95 •  select max(esal),min(esal) from employee;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

max(esal)	min(esal)
90000	55000

15. Give all employee who joined before 2022 a 10% raise :-

Result Grid | Filter Rows: | Edit: | Expo

eno	ename	jobtitle	jointdate	eadd	esal	dno
101	Manoj	Manager	2024	Mumbai	75000	10
102	Meera	Developer	2021	Pune	65000	20
103	Aakash	Analyst	2023	Delhi	55000	20
104	Neha	Intern	2026	Mumbai	NULL	30
105	Rahul	Manager	2020	Bangalore	90000	40
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
97 •  update employee
98    set esal=1.1*esal
99    where jointdate<2022;
100
101 •  select * from employee;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

eno	ename	jobtitle	jointdate	eadd	esal	dno
101	Manoj	Manager	2024	Mumbai	75000	10
102	Meera	Developer	2021	Pune	71500	20
103	Aakash	Analyst	2023	Delhi	55000	20
104	Neha	Intern	2026	Mumbai	NULL	30
105	Rahul	Manager	2020	Bangalore	99000	40
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Name :- Aaryan N. Raorane Roll no :- 5004 Div :- 5
branch :- AI & DS SAP ID :- 60018250070

accepted

Experiment No 4

1) Select :

```
select column1,column2
from table_name;
```

Retrieves selected columns from a table

2) where :

```
Select *
from tablename
where condition;
```

filters rows according to a condition

3) Comparison Operators :

```
select *
from tablename
where column_name operator values
```

operators => =, >, <, >=, <= and !=
filters using the above operators

4) AND :

```
select * from tablename
where condition1 AND condition2;
```

returns the row satisfying both conditions.

Sundaram
FOR EDUCATIONAL USE

5) OR :-

```
Select *
from tablename
where condition1 OR condition2;
```

returns rows satisfying at least one condition

6) NOT :-

```
select *
from tablename
where not condition;
```

returns rows that do not satisfy the condition.

7) Between :-

```
select *
from tablename
where columnname Between value1 and value2;
```

selects values within an inclusive range.

8) IN :-

```
select *
from tablename
where columnname IN (value1, value2, value3);
```

checks whether a column matches any value from a list

Sundaram
FOR EDUCATIONAL USE

9) Not In :-

```
select *
from tablename
where columnname Not In (value1, value2);
```

Selects rows whose column value is not in the specified list.

10) Like :-

```
select *
from tablename
where columnname Like pattern;
```

Searches text using pattern; % represents zero or more characters and _ represent one character

11) distinct :-

```
select distinct columnname
from tablename;
```

returns only unique values from a column.

12) Aggregate functions :-

```
select aggregate function(column name)
from tablename.
```

Sundaram
FOR EDUCATIONAL USE

calculates a summary using functions such as count(), sum(), avg(), Min(), or Max().

13) count :

```
select count(column name)
from tablename;
```

counts non-NULL values in a column

14) sum :

```
select sum(column name)
from tablename;
```

calculates the total of numeric values.

15) avg :

```
select avg(column name)
from tablename;
```

calculates the average of numeric values

16) Min :

```
select Min(column name)
from tablename;
```

returns the smallest value in a column

Sundaram
FOR EDUCATIONAL USE

17) Max:

```
select Max(column_name)
from table_name;
```

returns the largest value in a column.

18) Group by:

```
select column_name, count(*)
from table_name
Group by column_name;
```

Groups rows with same value and calculates a summary for each group.

19) Having:

```
select column_name, count(*)
from table_name
Group by column_name
having count(*) >= 2;
```

filters groups after Group by has been applied

20) AS alias:

```
Select column_name AS alias_name from table_name;
```

Give a temporary name to a column in the query result.

Sundaram
FOR EDUCATIONAL USE

21) Order By :

```
Select *
from tablename
order by column_name ;
```

sorts the result in ascending order by default

```
select *
from tablename
order by column_name DESC ;
```

sorts the result in descending order.

Sundaram FOR EDUCATIONAL USE